

Ejercicios algoritmos

Juan Marcos Aguirre Simionato

2026-04-08

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

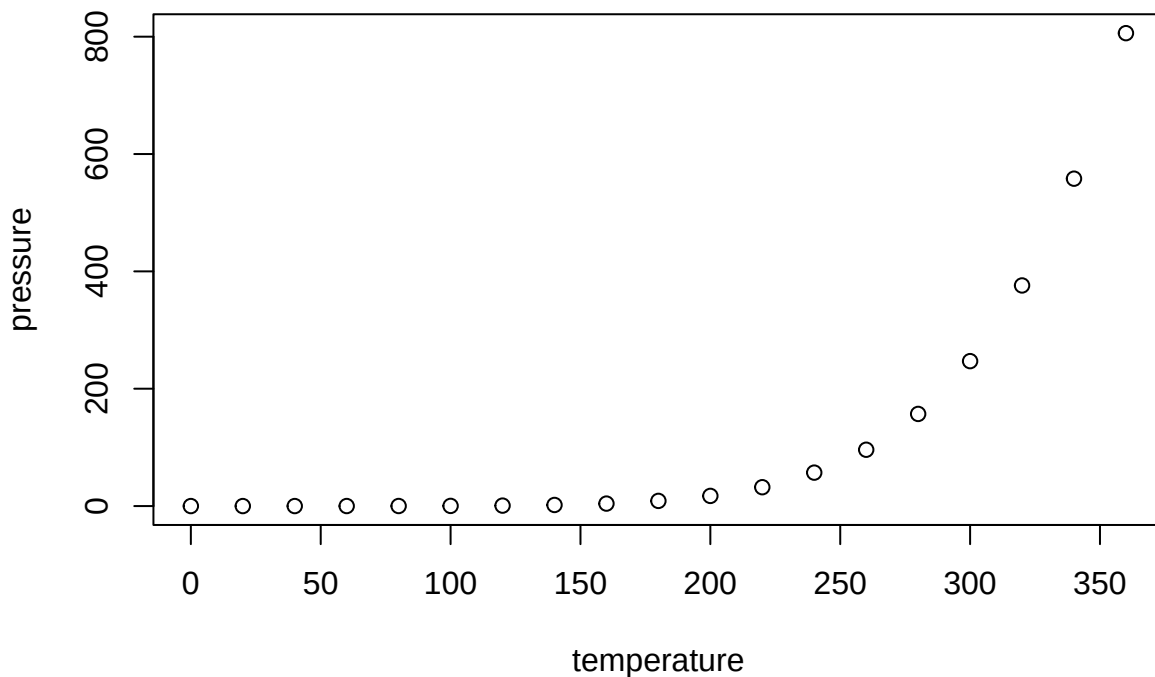
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   : 2.00
## 1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##   Mean  :15.4    Mean   : 42.98
## 3rd Qu.:19.0    3rd Qu.: 56.00
##   Max.  :25.0    Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

##Ejercicio 1

Consigna: las últimas 3 de mi DNI son 041, crear una variable que tenga ese n°.

```
DNI=041
```

Consigna: crear un vector que tenga todos los números enteros desde el 1 hasta el 041.

```
secuencia_dni <- (1:41)
secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41
```

##Ejercicio 2

Calcular la suma de todos los valores del vector `secuencia_dni`

```
total <- 0
valor_final <- length(secuencia_dni)
for (i in 1:valor_final)
  total <- total + i
total
```

```
## [1] 861
```

##Ejercicio 3

Consigna: Repetir el ejercicio anterior en Python.

```
suma=0
for i in range(1,41):
    suma+= i
    print(suma)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
```

```
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
```

##Ejercicio 4

¿Cuanto tarda en correr el código del ejercicio 2?

```
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 8.405e+16
```

```
final <- Sys.time()
final-inicio
```

```
## Time difference of 6.546679 secs
```

##Ejercicio 5

Consigna: repetir el ejercicio anterior utilizando la cabeza de Gauss.

```
inicio <- Sys.time()

valor_final <- 10000000 * length(secuencia_dni)

total <- valor_final * (valor_final + 1) / 2

total
```

```
## [1] 8.405e+16
```

```
final <- Sys.time()
final - inicio
```

```
## Time difference of 0.00270009 secs
```

##Ejercicio 6

Consigna: aprende a usar tictoc

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
```

```
## [1] "dormire una siestita..."
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
```

```
## [1] "...suena el despertador"
## [1] "...suena el despertador"
toc()
```

```
## sleeping: 2.005 sec elapsed
```

```
##Ejercicio 7
```

```
inicio_for <- Sys.time()
```

```
A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}
```

```
final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

```
# Generar con SEQ
inicio_seq <- Sys.time()
```

```
B <- seq(2, 100000, by = 2)
```

```
final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq
```

```
# Resultados
tiempo_for
```

```
## Time difference of 0.005966187 secs
```

```
tiempo_seq
```

```
## Time difference of 0.001655817 secs
```

```
##Ejercicio 8
```

```
Consigna: serie de Fibonacci
```

```
# Generar Fibonacci hasta superar 1.000.000
```

```
fibonacci <- c(0,1)
iteraciones <- 2
```

```
while (length(fibonacci) <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
```

```

    iteraciones <- iteraciones + 1
}

```

```
fibonacci
```

```

## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]   196418   317811   514229   832040  1346269

```

```
iteraciones
```

```
## [1] 32
```

```
##Ejercicio 9
```

Consigna: ordenamiento

```
library(microbenchmark)
```

```
# Generar muestra grande
```

```
set.seed(123)
```

```
x <- sample(1:100000, 20000)
```

```
# Función burbuja
```

```

burbuja <- function(x){
  n <- length(x)
  for (j in 1:(n-1)){
    for (i in 1:(n-j)){
      if (x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

```

```
# Benchmark
```

```

resultado <- microbenchmark(
  burbuja = burbuja(x),
  sort_R = sort(x),
  times = 5
)

```

```
resultado
```

```

## Unit: microseconds
##      expr      min       lq      mean     median      uq
## burbuja 32393366.31 32682979.77 3.291999e+07 33068594.159 33192475.769
## sort_R    769.74    772.65 8.322826e+02    811.631    843.341
##      max neval
## 33262546.905     5
##    964.051     5

```

```
##Ejercicio 10 Consigna: Potencia de Newton
```

```

# Método 1: FOR (ineficiente)
inicio1 <- Sys.time()
n <- 10000000
suma1 <- 0
for (i in 1:n) {
  suma1 <- suma1 + i
}

final1 <- Sys.time()
tiempo1 <- final1 - inicio1

# Método 2: Fórmula de Gauss (eficiente)
inicio2 <- Sys.time()

suma2 <- n * (n + 1) / 2

final2 <- Sys.time()
tiempo2 <- final2 - inicio2

# Resultados
tiempo1

```

```
## Time difference of 0.1596704 secs
```

```
tiempo2
```

```
## Time difference of 0.001260519 secs
```